



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG VỚI UML

BỘ MÔN CÔNG NGHỆ PHẦN MỀM



Giới thiệu nội dung học phần

Bài 1: Giới thiệu quy trình ICONIX Process

Bài 2: Mô hình hóa miền (Domain modeling)

Bài 3: Mô hình hóa Use Case (Use Case Modeling)

Bài 4: Requirements Review – Kiểm duyệt yêu cầu → **Kiểm tra TX1**

Bài 5: Robustness Analysis — Cầu nối từ Use Case đến Thiết kế

Bài 6: Preliminary Design Review — Kiểm duyệt thiết kế sơ bộ → **Kiểm tra TX 2**

Bài 7: Technical Architecture — Kiến trúc kỹ thuật

Bài 8: Sequence Diagrams

Bài 9: Critical Design Review (CDR) → **Kiểm tra TX3**



TẦM QUAN TRỌNG CỦA MÔN HỌC

Môn học này có quan trọng không → Có. Vì:

- Nó là tài liệu bắt buộc trong đề tài, đồ án, dự án...
- Nó là cơ sở để cho các lập trình viên viết code
- Nó là tài liệu để cho các mô hình AI hiện đại sinh mã nguồn, chương trình...



BÀI 7: TECHNICAL ARCHITECTURE (TA)

Về kiến thức

-  Khái niệm Technical Architecture (TA)
-  Vai trò trong ICONIX Process
-  Top 10 Guidelines
-  Phân tầng kiến trúc
-  IoC / DI – Testability – Security
-  Các lỗi kiến trúc phổ biến

Về kỹ năng

-  Hiểu vai trò Technical Architecture
-  Nắm vững mô hình phân tầng
-  Áp dụng IoC / DI đúng cách
-  Đảm bảo testability & security
-  Tránh 10 lỗi kiến trúc phổ biến
-  Chuẩn bị cho sequence diagram

Định nghĩa Technical Architecture



Cầu nối từ “what” → “how”



Bản thiết kế kỹ thuật tổng thể



Thống nhất cấu trúc hệ thống



Xác định component chính



Hợp đồng giữa các lập trình viên



Vai trò trong ICONIX



Sau PDR,
trước Sequence
Diagram



Xác định cấu
trúc tầng kỹ
thuật



Liên kết logic
– framework –
hạ tầng



Giảm xung
đột & lỗi thiết kế



Cửa ngõ
sang Detailed
Design



Lợi ích của Technical Architecture



Giữ tính thống nhất



Giảm coupling giữa tầng



Hỗ trợ kiểm thử & bảo trì



Dễ mở rộng & chia nhóm



Ngăn “spaghetti code”



Top 10 Guidelines (Tổng quan)

10 Giữ kiến trúc đơn giản – rõ ràng

 Phân tầng & tách trách nhiệm

 Dùng Component / Deployment Diagram

 Liên kết framework thực tế

 Chia sẻ & kiểm duyệt sớm

#10 Giữ kiến trúc đơn giản



“Simple but not simpler”



Dễ hiểu, dễ bảo trì



Đủ tầng, không dư tầng



Phù hợp quy mô dự án



Tránh “vẽ để đẹp”

#9 Phân tách rõ trách nhiệm



UI: giao
tiếp người
dùng



Business:
xử lý logic



Persistence:
lưu trữ dữ liệu



Không
chồng chéo
tầng



Tách rõ “ai
làm gì”



#8: Thực hiện qua Component Diagram



Dùng UML Component Diagram



Mỗi tầng = nhóm component



Biểu diễn dependency



Rõ interface và cổng kết nối



Làm nền cho triển khai thật

#7: Gắn liền với Framework thực tế



Chọn framework sớm



Gắn mô hình với công nghệ



Ví dụ: Spring MVC / ASP.NET



Giảm “vẽ trên giấy”



Chạy được, không chỉ đẹp



#6: Giữ một chiều phụ thuộc



Không vòng lặp giữa tầng



Hướng phụ thuộc: UI $\rightarrow$ Logic $\rightarrow$ Data



Không logic ngược



Tăng khả năng kiểm thử



Dễ thay thế module

#5: Đảm bảo tính mở rộng (Scalability)



Module
độc lập



Dễ thêm
service mới



Có
caching,
async



Tách
frontend—
backend



Hỗ trợ
cloud
deployment

#4: Dễ kiểm thử (Testability)



Mỗi tầng
test độc lập



Dùng DI
để mock



Giảm
dependency
cứng



Unit test
= tự nhiên



Phát hiện
lỗi sớm

#3: Đảm bảo an toàn (Security)



Kiểm soát truy cập



Xác thực & phân quyền



Validation & sanitization



Bảo mật giao tiếp



Không hard-code key



#2: Kiểm duyệt tập thể

 Review kiến trúc nhóm

 Dev + Architect + Tester

 Phát hiện sớm xung đột

 Tài liệu hóa quyết định

 Đồng thuận trước khi code



#1: Cập nhật và sống cùng dự án

 Kiến trúc = tài liệu sống

 Cập nhật sau mỗi sprint

 Sync với domain & code

 Dễ bảo trì lâu dài

 Không “đóng băng”

Mô hình phân tầng cơ bản



Presentation Layer



Business Logic Layer



Data Access Layer

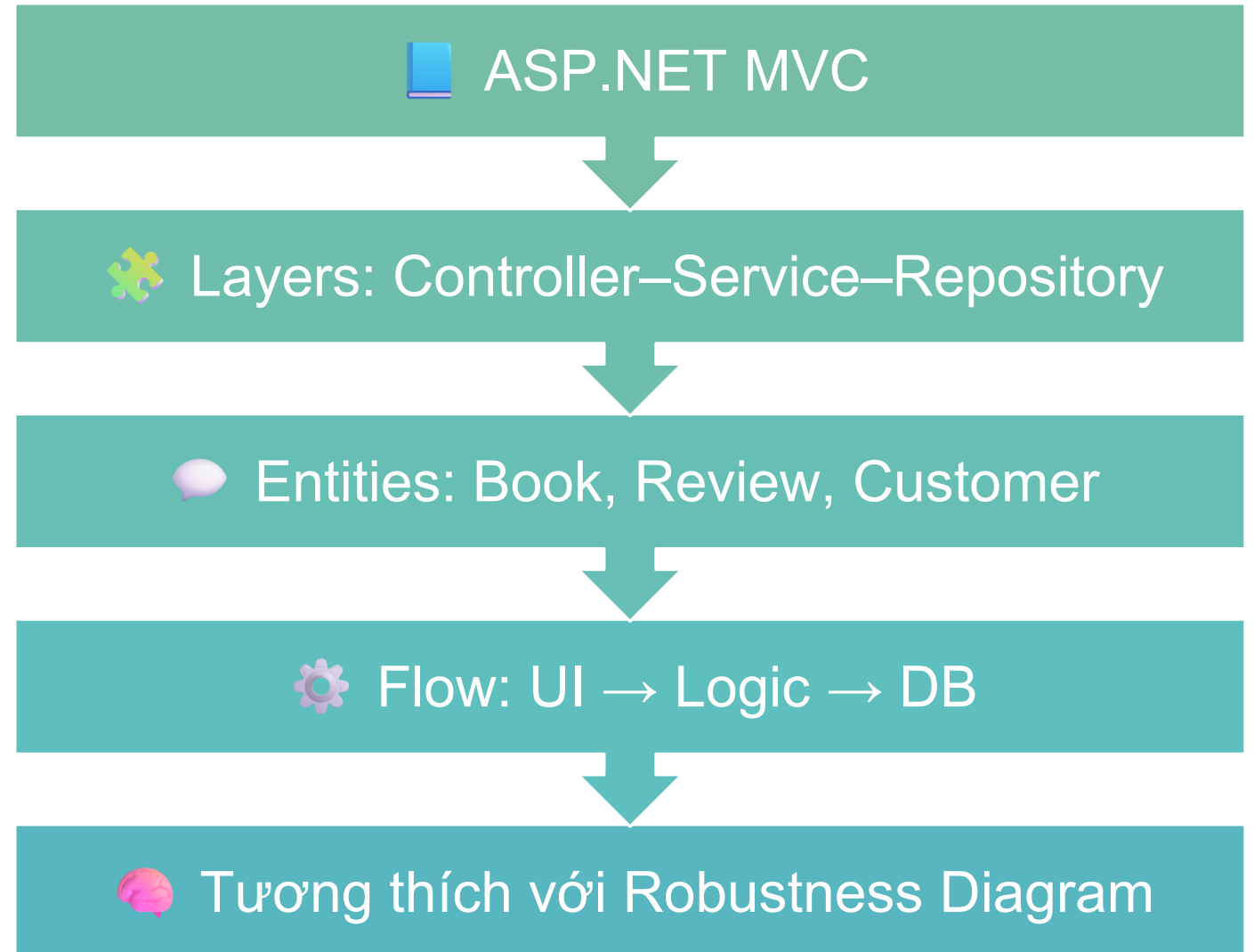


Communication giữa tầng



Một chiều duy nhất

Ví dụ Internet Bookstore



Inversion of Control (IoC)



Đảo chiều điều khiển



Container quản lý dependency



Giảm coupling



Dễ test, dễ thay thế



Áp dụng ở Service & DAO



Dependency Injection (DI)

 Cơ chế
hiện thực IoC

 Inject qua
constructor
hoặc interface

 Dễ mock
khi test

 Thay đổi
implementation
linh hoạt

 Áp dụng
toàn dự án

Testability trong TA



Test theo tầng



Mock dependencies



Sử dụng unit & integration test



Áp dụng CI/CD



Đo coverage tự động



Security trong TA



Authentication & Authorization



Bảo vệ API endpoints



Input validation



Mã hóa dữ liệu



Áp dụng OWASP principles

10 Lỗi Kiến trúc Phổ biến

✗ Không phân tầng rõ

↺ Phụ thuộc ngược chiều

🧩 Thiếu DI

💬 Lẫn logic vào UI

⚙️ Không kiểm tra performance



Liên hệ với ICONIX tổng thể



TA sau PDR



Chuẩn bị Sequence Diagram



Định hình design pattern



Kết nối code & model



Nền tảng cho DDD / EF Core

Tổng kết

-  TA = Bản đồ kỹ thuật sống
-  Đơn giản – rõ ràng – khả thi
-  IoC/DI là nền tảng
-  Testability & Security = bắt buộc
-  Liên tục cập nhật

Câu hỏi ôn tập

? Technical Architecture là gì?

? Trình bày 3-tier model.

? IoC và DI khác nhau thế nào?

? Vì sao testability quan trọng?

? Liệt kê 3 lỗi kiến trúc phổ biến.



Bài tập thực hành

 Thiết kế TA cho Use Case “Search Book”

 Áp dụng MVC / 3-tier

 Thêm IoC & DI

 Viết test cho Controller

 Review nhóm 3 người